

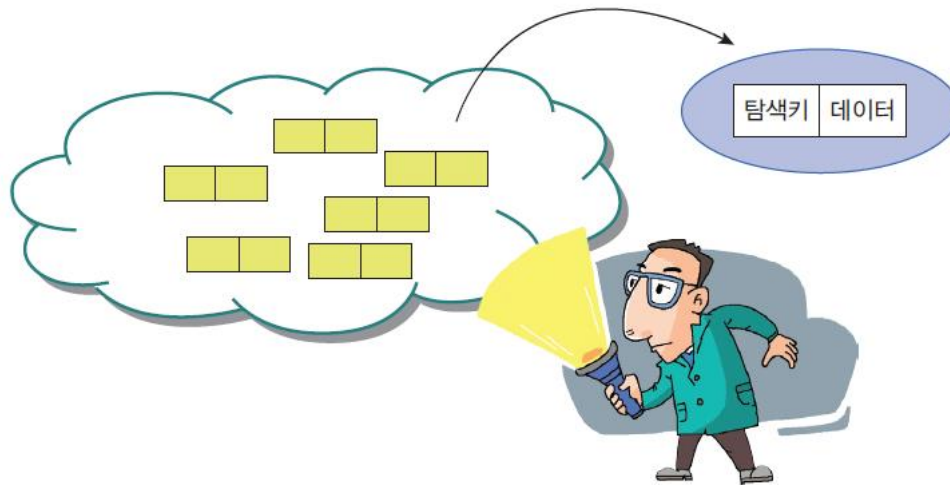
탐색과 해싱

탐색의 정의

- 탐색 (search)은 여러 개의 자료들 중에서 원하는 자료를 찾는 작업
 - 컴퓨터가 가장 많이 하는 작업 중의 하나 (웹 탐색)
 - 탐색을 효율적으로 수행하는 것이 컴퓨터의 성능에 큰 영향을 미침
- 탐색키 (search key) : 항목과 항목을 구별해주는 키 (key)
- 대표적인 탐색 방법 :
 - 순차 탐색,
 - 이진 탐색,
 - 이진 탐색 트리,
 - 해싱

탐색키

- **탐색키**(search key)
 - 항목과 항목을 구별해주는 키(key)
- 탐색을 위하여 사용되는 자료 구조
 - 배열, 연결 리스트, 트리, 그래프 등



순차 탐색

- 탐색 방법 중에서 가장 간단하고 직접적인 탐색 방법
- 정렬되지 않은 배열을 처음부터 마지막까지 하나씩 검사하는 방법
- 비교 횟수
 - 최선 탐색: 1번 비교
 - 최악 탐색: n 번 비교
- 시간 복잡도: $O(n)$

순차 탐색

• 8을 찾는 경우

(1) $9 \neq 8$ 이므로 탐색 계속

9	5	8	3	7
---	---	---	---	---

(2) $5 \neq 8$ 이므로 탐색 계속

9	5	8	3	7
---	---	---	---	---

(3) $8 = 8$ 이므로 탐색 성공

9	5	8	3	7
---	---	---	---	---

(a) 탐색 성공의 경우

• 2를 찾는 경우

(1) $9 \neq 2$ 이므로 탐색 계속

9	5	8	3	7
---	---	---	---	---

(2) $5 \neq 2$ 이므로 탐색 계속

9	5	8	3	7
---	---	---	---	---

(3) $8 \neq 2$ 이므로 탐색 계속

9	5	8	3	7
---	---	---	---	---

(4) $3 \neq 2$ 이므로 탐색 계속

9	5	8	3	7
---	---	---	---	---

(5) $7 \neq 2$ 이므로 탐색 계속

9	5	8	3	7
---	---	---	---	---

(6) 더 이상 항목이 없으므로
탐색 실패

(b) 탐색 실패의 경우

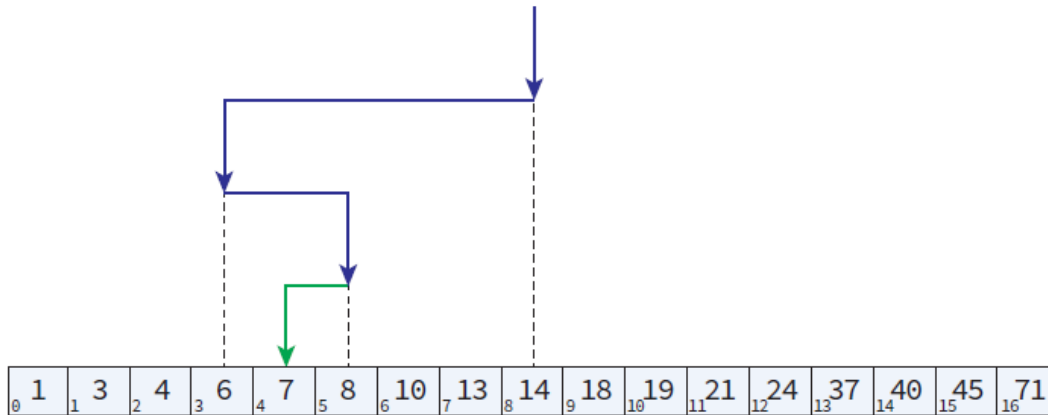
순차 탐색

```
define max_std 20
struct {
    no, name, tel
} std_info
std_info student[max_std]
```

1. Algorithm S_search(student, key, n)
2. Input: 학생 정보 구조체 배열(student), 학번(key), 학생 수(n)
3. Output: key 값과 일치하는 학생 정보, 존재하지 않을 경우 Null
4. for i <- 0 to n-1 do
5. if (key = student[i].no) return student[i]
6. return Null

이진탐색

- 선행 조건 : 정렬(오름차순)
- 정렬된 배열의 중앙에 있는 값을 조사하여 찾고자 하는 항목이 왼쪽 또는 오른쪽 부분 배열에 있는지를 알아내어 탐색의 범위를 반으로 줄여가며 탐색 진행
- (예) 10억 명중에서 특정한 이름 탐색
 - 이진탐색 : 단지 30번의 비교 필요
 - 순차 탐색 : 평균 5억 번의 비교 필요



이진탐색

(표 12.1) 이진탐색으로 25를 찾는 과정

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
(1)	10	15	20	25	30	35	40	45	50	55	60
(2)	10	15	20	25	30	35	40	45	50	55	60
(3)	10	15	20	25	30	35	40	45	50	55	60

이진탐색

• 5를 탐색하는 경우

7과 비교

1	3	5	6	7	9	11	20	30
---	---	---	---	---	---	----	----	----

$5 < 7$ 이므로 앞부분만을 다시 탐색

1	3	5	6
---	---	---	---

5를 3과 비교

1	3	5	6
---	---	---	---

$5 > 3$ 이므로 뒷부분만을 다시 탐색

5	6
---	---

$5 == 5$ 이므로 탐색 성공

5	6
---	---

(a) 탐색이 성공하는 경우

• 2를 탐색하는 경우

7과 비교

1	3	5	6	7	9	11	20	30
---	---	---	---	---	---	----	----	----

$2 < 7$ 이므로 앞부분만을 다시 탐색

1	3	5	6
---	---	---	---

2를 3과 비교

1	3	5	6
---	---	---	---

$2 < 3$ 이므로 앞부분만을 다시 탐색

1

$2 > 1$ 이므로 뒷부분만을 다시 검색

1

더 이상 남은 항목이 없으므로 탐색 실패

(b) 탐색이 실패하는 경우

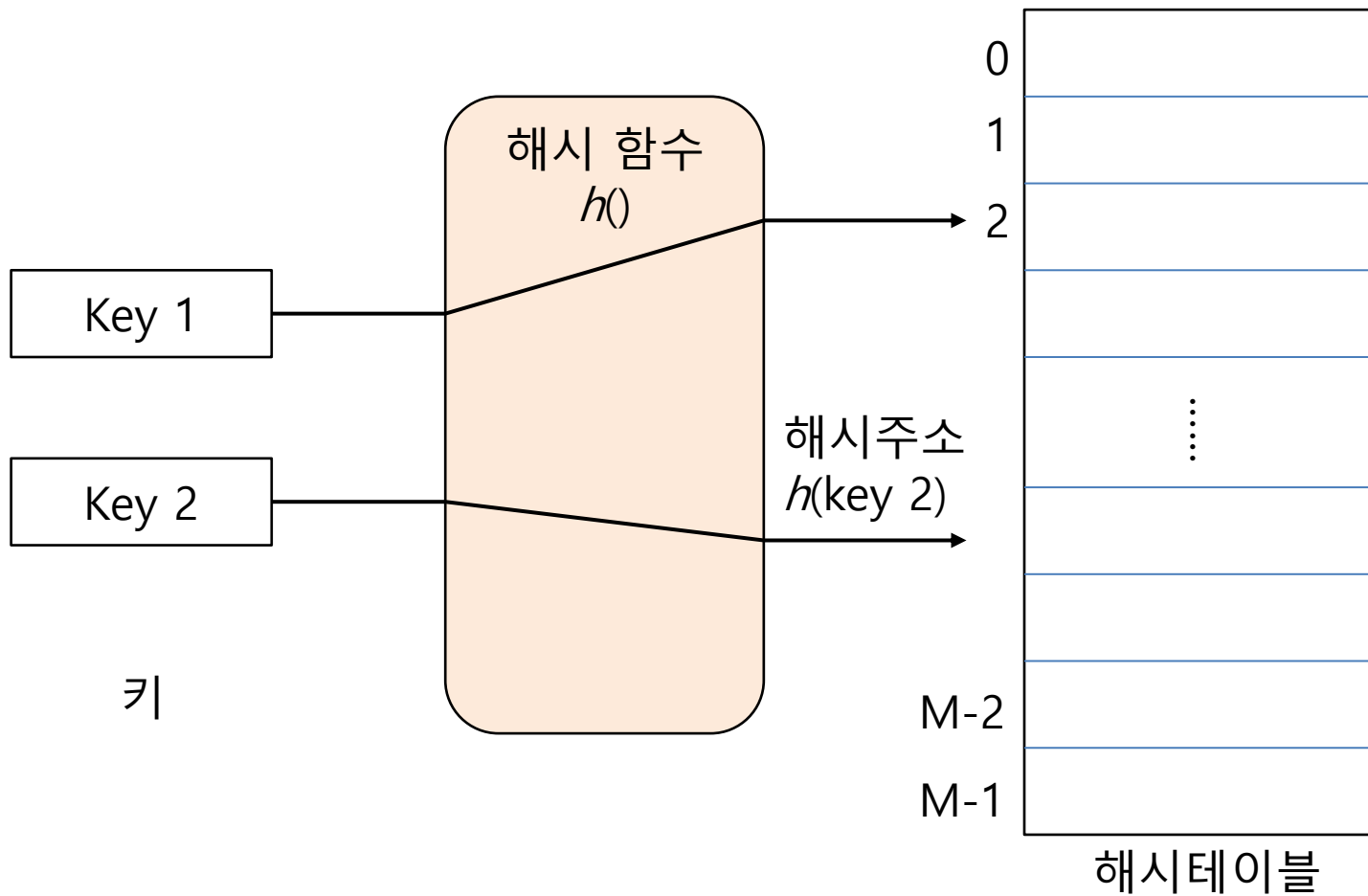
이진탐색

1. Algorithm B_search(student, key, n)
2. Input: 학생정보 구조체 배열(student), 학번(key), 학생 수(n)
3. Output: key 값과 일치하는 학생정보, 존재하지 않을 경우 Null
4. left $\leftarrow$ 0
5. right $\leftarrow$ n-1
6. while (left $\leq$ right) do {
7. mid $\leftarrow$ (left + right) / 2
8. if (key = student[mid].no) return student[i]
9. if (key > student[mid].no)
10. left $\leftarrow$ mid + 1
11. else
12. right $\leftarrow$ mid - 1
13. }
14. return Null

해시탐색

- 지금까지의 탐색 방법은 키 값을 비교
- 해싱은 키 값에 산술 연산을 적용하고, 그 연산 결과가 찾는 레코드가 있는 위치 (인덱스)인 탐색 방법
- 해시 함수 (hash function)는 해싱에서 산술 연산을 적용하는 부분
- 해시 테이블 (hash table)은 해시 주소를 인덱스로 하여 접근되는 배열형태의 자료 구조

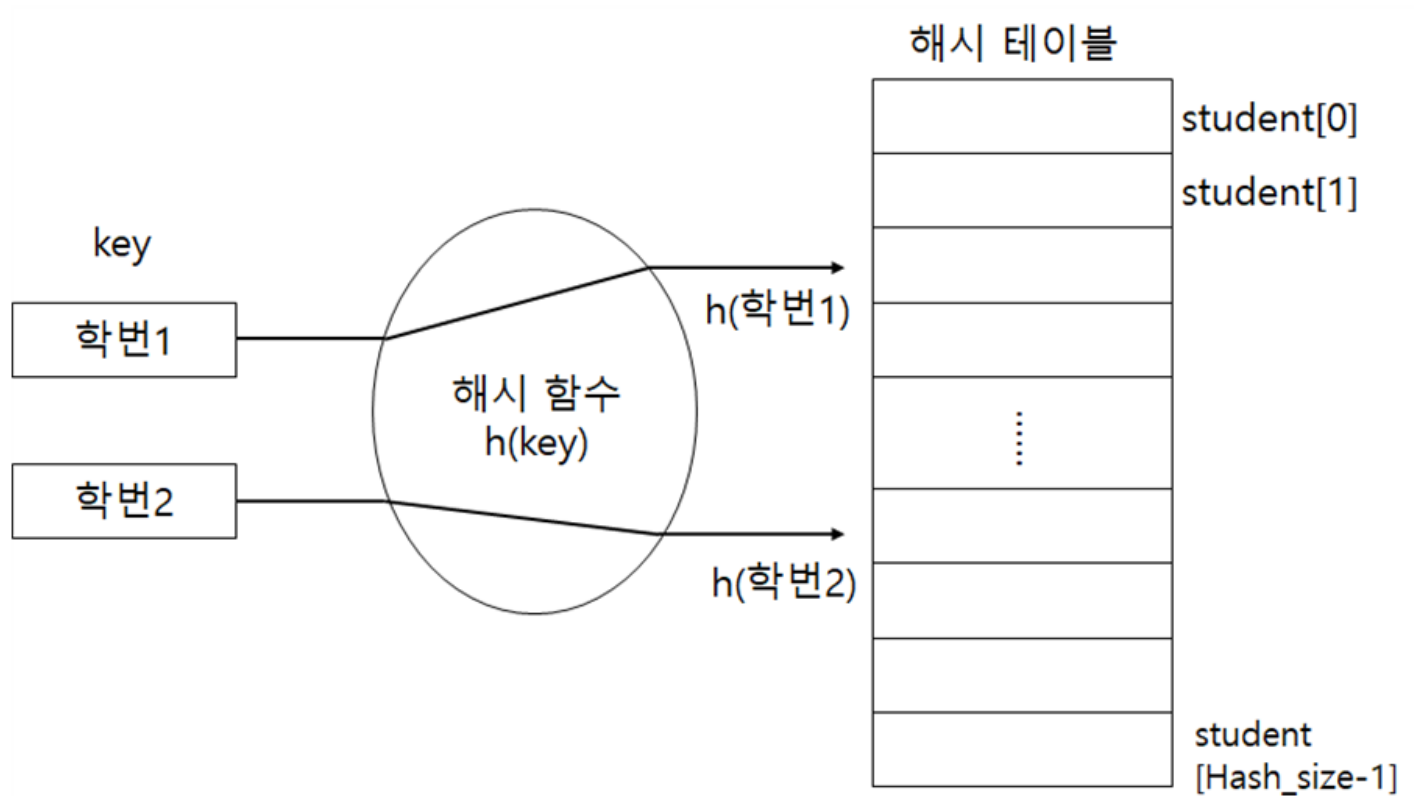
해시탐색



해시탐색

```
define Hash_size 11
struct {
    no, name, tel
} std_info
std_info student[Hash_size]
```

해시탐색



해시탐색

$h(\text{key}) = (\text{key에 포함된 각 디지트의 아스키코드의 합}) \% \text{Hash_size}$
 $\rightarrow [0, \text{Hash_size}-1]$

“128”의 아스키코드 합 = $49 + 50 + 56 = 155$

$h(\text{“128”}) = 155 \% 11 = 1$

해시탐색

학번	이름	전화번호
100	Kim	111
128	Hong	333
115	Jung	222
218	Cha	444
634	Park	666
259	Chu	777
511	Gang	888

해시탐색

$$h("100") = (49+48+48) \% 11 = 2$$

$$h("128") = (49+50+56) \% 11 = 1$$

$$h("115") = (49+49+53) \% 11 = 8$$

$$h("218") = (50+49+56) \% 11 = 1 \text{ (충돌)}$$

$$h("634") = (54+51+52) \% 11 = 3$$

$$h("259") = (50+53+57) \% 11 = 6$$

$$h("511") = (53+49+49) \% 11 = 8 \text{ (충돌)}$$

해시탐색

- 선형조사법

$(h(key)+1) \% Hash_size, (h(key)+2) \% Hash_size,$
 $(h(key)+3) \% Hash_size, (h(key)+4) \% Hash_size, \dots$

해시탐색

$$h("100") = (49+48+48) \% 11 = 2$$

$$h("128") = (49+50+56) \% 11 = 1$$

$$h("115") = (49+49+53) \% 11 = 8$$

$$h("218") = (50+49+56) \% 11 = 1$$

$$h("634") = (54+51+52) \% 11 = 3$$

$$h("259") = (50+53+57) \% 11 = 6$$

$$h("511") = (53+49+49) \% 11 = 8$$

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
no											
name											
tel											

해시탐색

(표 12.2) 크기 11을 갖는 해시 테이블(student)의 예

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
no		128	100	218	634		259		115	511	
name		Hong	Kim	Cha	Park		Chu		Jung	Gang	
tel		333	111	444	666		777		222	888	

해시 탐색

1. Algorithm Hash_function(key, Hash_size)
2. Input: 키 값(key), 해시 크기(Hash_size)
3. Output: 해시 주소 값
4. len <- string_lengh(key)
5. sum <- 0
6. for i <- 0 to len-1 ro
7. sum <- sum + key[i]
8. hash_addr <- sum % Hash_size
9. return hash_addr

해시 탐색

```
1. Algorithm Sinfo_create(student, Hash_size)
2. Input: 해시 테이블(student), 해시 크기(Hash_size)
3. Output: 학생 정보가 저장된 해시 테이블
4. for i <- 0 to Hash_size-1 do {
5.     student[i].no <- Null
6.     student[i].name <- Null
7.     student[i].tel <- Null }
8. open student_info.txt
9. while (!end_of_file) do {
10.     read no, name, tel
11.     haddr <- Hash_function(no, Hash_size)
12.     index <- haddr
13.     while (student[index].no != Null) do {
14.         if (student[index].no = no) {
15.             print error: duplicated key
16.             return }
17.         index <- (index+1) % Hash_size
18.         if (index = haddr) {
19.             print error: hash full
20.             return }
21.         }
22.     student[index].no <- no
23.     student[index].name <- name
24.     student[index].tel <- tel
25. }
26. return student
```

해시 탐색

1. Algorithm Sinfo_search(student, key, Hash_size)
2. Input: 해시 테이블(student), 해시 크기(Hash_size), 키 값(key)
3. Output: 키 값(학번)과 일치하는 학생 정보, 키 값이 없을 경우 NULL
4. haddr $\leftarrow$ Hash_function(key, Hash_size)
5. index $\leftarrow$ haddr
6. while (true) do {
7. if (student[index].no = key)
8. return student[index]
9. if (student[index].no = Null)
10. break
11. index $\leftarrow$ (index+1) % Hash_size
12. if (index = haddr)
13. break
14. }
15. return Null